Laporan Tugas Besar 1 IF3270 Pembelajaran Mesin
Semester II Tahun Akademik 2025/2026
Implementasi Algoritma Pembelajaran Mesin

Disusun Oleh:

Anas Fathurrahman	13222033
William Gerald Briandelo	13222061

Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

Deskripsi Persoalan

Tujuan

Tugas Besar I pada kuliah IF3270 Pembelajaran Mesin agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan *Feedforward Neural Network* (FFNN). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan *FFNN from scratch*.

Spesifikasi

Implementasikan suatu modul FFNN yang memenuhi ketentuan-ketentuan berikut:

- FFNN yang diimplementasikan dapat **menerima jumlah neuron dari tiap layer** (termasuk input layer dan output layer).
- FFNN yang diimplementasikan dapat **menerima fungsi aktivasi dari tiap layer**. Pilihan fungsi aktivasi yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
Linear	$Linear(x) = x$
ReLU	$ReLU(x) = \max(0, x)$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent (tanh)	$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$

- FFNN yang diimplementasikan dapat **menerima fungsi loss** dari model tersebut. Pilihan loss function yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
MSE	$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$
Binary Cross-Entropy	$$\mathcal{L}_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i))$$ y_i = Actual binary label (0 or 1) $\hat{y}_i$ = Predicted value of y_i n = Batch size
Categorical Cross-Entropy	$$\mathcal{L}_{CCE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}(y_{ij} \log \hat{y}_{ij})$$ y_{ij} = Actual value of instance i for class j $\hat{y}_{ij}$ = Predicted value of y_{ij} C = Number of classes n = Batch size

    - Catatan:
        - Binary cross-entropy merupakan kasus khusus categorical cross-entropy dengan kelas sebanyak 2.
        - Log yang digunakan merupakan logaritma natural (logaritma dengan basis e).
- Terdapat mekanisme untuk **inisialisasi bobot** tiap neuron (termasuk bias). Pilihan metode inisialisasi bobot yang harus diimplementasikan adalah sebagai berikut:
    - **Zero initialization**
    - Random dengan distribusi **uniform**.
        - Menerima parameter lower bound (batas minimal) dan upper bound (batas maksimal).
        - Menerima parameter seed untuk reproducibility.
    - Random dengan distribusi **normal**.
        - Menerima parameter mean dan variance.
        - Menerima parameter seed untuk reproducibility.
- Instance model yang diinisialisasikan harus bisa **menyimpan bobot** tiap neuron (termasuk bias).

- Instance model yang diinisialisasikan harus bisa **menyimpan gradien bobot** tiap neuron (termasuk bias).
- Instance model memiliki method untuk **menampilkan distribusi bobot** dari tiap layer.
    - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk **menampilkan distribusi gradien bobot** dari tiap layer.
    - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk ***save*** dan ***load***.
- Model memiliki implementasi **forward propagation** dengan ketentuan sebagai berikut:
    - Dapat menerima input berupa **batch**.
- Model memiliki implementasi **backward propagation** untuk menghitung perubahan gradien:
    - Dapat menangani perhitungan perubahan gradien untuk input data **batch**.
    - Gunakan konsep **chain rule** untuk menghitung gradien tiap bobot terhadap loss function.
    - Berikut merupakan **turunan pertama** untuk setiap fungsi aktivasi:

Nama Fungsi Aktivasi	**Turunan Pertama**
Linear	$\frac{d\big(Linear(x)\big)}{dx} = 1$
ReLU	$\frac{d\big(ReLU(x)\big)}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$
Sigmoid	$\frac{d\big(\sigma(x)\big)}{dx} = \sigma(x)\big(1 - \sigma(x)\big)$
Hyperbolic Tangent (tanh)	$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^{x} - e^{-x}}\right)^{2}$

Softmax	Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $$\frac{d(softmax(\vec{x})_i)}{d\vec{x}} = \begin{bmatrix} \frac{\partial(softmax(\vec{x})_1)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(softmax(\vec{x})_n)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_n)}{\partial x_n} \end{bmatrix}$$ Dimana untuk $i, j \in \{1, \dots, n\}$, $$\frac{\partial(softmax(\vec{x})_i)}{\partial x_j} = softmax(\vec{x})_i(\delta_{i,j} - softmax(\vec{x})_j)$$ $$\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

- Berikut merupakan **turunan pertama** untuk setiap fungsi loss terhadap bobot suatu FFNN (lanjutkan sisanya menggunakan chain rule):

Nama Fungsi Loss	Definisi Fungsi
MSE	$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial W}$
Binary Cross-Entropy	$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\hat{y}_i - y_i}{\hat{y}_i(1-\hat{y}_i)}\frac{\partial \hat{y}_i}{\partial W}$
Categorical Cross-Entropy	$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{y_{ij}}{\hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W}$

- Model memiliki implementasi metode **regularisasi L1** dan **L2**.
- Model memiliki implementasi **weight update** dengan menggunakan **gradient descent** untuk memperbarui bobot berdasarkan gradien yang telah dihitung, berikut persamaannya:

$$W_{new} = W_{old} - \alpha\left(\frac{\partial \mathcal{L}}{\partial W_{old}}\right)$$

α = Learning rate

- Implementasi untuk pelatihan model harus memenuhi ketentuan berikut:
  - Dapat menerima parameter berikut:
    - Batch Size
    - Learning Rate
    - Jumlah Epoch
    - Verbose
- Verbose 0 berarti tidak menampilkan apa-apa selama pelatihan.

- Verbose 1 berarti hanya menampilkan progress bar beserta dengan kondisi training loss dan validation loss saat itu.
    - Proses pelatihan mengembalikan **histori dari proses pelatihan** yang berisi **training loss** dan **validation loss tiap epoch**

Pengujian

Gunakan dataset berikut untuk menguji model: global_student_placement_and_salary.

Pengujian dilakukan di **file .ipynb terpisah**.

1. **Hyperparameter**

   Analisis pengaruh beberapa **hyperparameter** sebagai berikut:

   - Pengaruh **depth (banyak layer)** dan **width (banyak neuron per layer)**
     - Pilih 3 variasi kombinasi width (depth tetap) dan 3 variasi depth (width semua layer tetap).
     - Bandingkan hasil akhir prediksinya.
     - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
   - Pengaruh **fungsi aktivasi** hidden layer.
     - Tentukan base arsitektur (minimal 3 layer), pilih 1 layer sebagai layer pengetesan.
     - Lakukan eksperimen dengan menjalankan variasi base arsitektur dengan setiap fungsi aktivasi yang diimplementasikan **kecuali softmax** pada layer pilihan.
     - Bandingkan hasil akhir prediksinya.
     - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
     - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
   - Pengaruh **learning rate**.
     - Lakukan 3 variasi learning rate (nilainya dibebaskan).
     - Bandingkan hasil akhir prediksinya.
     - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
     - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
2. **Pengaruh Regularisasi**.
   - Lakukan eksperimen masing-masing 1 kali untuk model tanpa regularisasi, dengan regularisasi L1, dan dengan regularisasi L2.
   - Bandingkan hasil akhir prediksinya.
   - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
   - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
3. **Uji Perbandingan**.

Analisis perbandingan hasil prediksi dengan library sklearn MLP.

- Lakukan satu kali pelatihan dengan hyperparameter yang sama untuk kedua model.
- Hyperparameter yang digunakan dibebaskan.
- Bandingkan hasil akhir prediksinya saja.

Informasi Dataset

Pada Tugas Besar ini, Anda juga diminta untuk melakukan **analisis dan pemodelan** menggunakan dataset *Global Student Placement & Salary Dataset* menggunakan **FFNN** yang sudah kalian buat. Dataset ini berisi kumpulan data global mengenai performa akademik mahasiswa serta outcome penempatan kerja dan gaji setelah lulus. Dataset dapat diakses **di sini**. Berikut deskripsi dari tiap fitur yang ada pada dataset yang akan digunakan (**placement_status** adalah kolom yang perlu diprediksi):

Column	Description	Type
cgpa	Cumulative Grade Point Average (skala 4.0–10.0)	Float
backlogs	Jumlah mata kuliah yang belum lulus	Integer
college_tier	Tingkatan institusi	Categorical
country	Negara kelulusan	Categorical
university_ranking_band	Kategori peringkat global universitas	Categorical
internship_count	Jumlah internship yang telah diselesaikan	Integer
aptitude_score	Skor tes kemampuan kuantitatif	Integer
communication_score	Skor kemampuan komunikasi	Integer
specialization	Bidang akademik	Categorical
industry	Industri tempat bekerja	Categorical
internship_quality_score	Skor kualitas pengalaman internship	Float
placement_status	Status penempatan kerja	Categorical

Pembahasan

Penjelasan Implementasi

1. Class (Attribute and Method)

a. Class Activation

Kelas yang berisi kumpulan fungsi aktivasi yang tercantum di dokumen spesifikasi, serta dua bonus fungsi aktivasi tambahan, yaitu Leaky ReLU dan ELU. Kelas ini juga berisikan turunan pertama dari fungsi aktivasi, yang akan digunakan pada saat backpropagation

- Attribute: Tidak ada
- Method:
  - linear(x) : fungsi ini akan mengembalikan nilai x sesuai dengan yang diinput.
  - relu(x) : fungsi ini akan mengembalikan nilai 0 jika x bernilai negatif, serta mengembalikan nilai x jika bernilai tidak negatif.
  - sigmoid(x) : fungsi ini akan memetakan nilai x ke range 0 sampai 1, sesuai dengan persamaan $\frac{1}{1+e^{-x}}$, dengan menerapkan batas untuk mencegah overflow, yaitu pada x -500 hingga 500.
  - tanh(x) : fungsi ini akan memetakan nilai x ke range -1 sampai 1 sesuai dengan persamaan $\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$.
  - softmax(x) : fungsi ini akan mengubah nilai logits menjadi distribusi probabilitas per kelas. Pada implementasi di kode, diterapkan pengurangan dengan nilai maksimum untuk mencegah overflow.
  - leaky_relu(x, alpha) : fungsi ini bekerja mirip seperti relu(x), hanya saja, jika x negatif tidak langsung diberi nilai nol, tapi dikalikan dengan suatu nilai alpha.
  - elu(x, alpha) : fungsi ini bekerja mirip seperti relu(x), hanya saja, jika x negatif tidak langsung diberi nilai nol, tapi dijadikan untuk input persamaan $\alpha * (e^{x} - 1)$.
- Method Turunan, method ini digunakan untuk metode backpropagation, yang membutuhkan turunan pertama dari fungsi aktivasi:
  - linear_deriv(x) : fungsi ini akan mengembalikan array berisikan nilai 1.

- relu_deriv(x) : fungsi ini akan mengembalikan nilai 1 jika x positif, dan 0 selain itu.
- sigmoid_deriv(x) : fungsi ini akan mengembalikan nilai sigmoid(x)*(1-sigmoid(x)).
- tanh_deriv(x) : fungsi ini akan mengembalikan nilai 1-$\tanh^2$(x).
- leaky_relu_deriv(x, alpha) : fungsi ini akan mengembalikan nilai 1 untuk x positif, dan alpha untuk x negatif.
- elu_deriv(x, alpha) : fungsi ini akan mengembalikan nilai 1 untuk x positif, dan elu(x) + alpha untuk x negatif.

b. Class Loss

Kelas yang berisi kumpulan fungsi Loss beserta turunan pertamanya.

- Atribut: Tidak ada
- Method:
  - mse(y_true, y_pred): fungsi ini akan menghitung mean squared error, yang menghitung rata-rata dari kuadrat selisih antara nilai asli dan nilai tebakan. MSE dapat dinyatakan dengan persamaan $\frac{1}{n}\sum(y_{true} - y_{pred})$.
  - bce(y_true, y_pred) : fungsi akan menghitung binary cross-entropy, yang digunakan untuk kasus klasifikasi biner yang berpasangan dengan fungsi sigmoid. Fungsi ini menggunakan logaritma untuk menghukum tebakan yang sangat meleset dengan penalti yang eksponensial. Lalu, untuk mencegah terjadi overflow, nilai dibatasi menjadi (1e-15, 1 - 1e-15). BCE dapat dirumuskan sebagai berikut
    $$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$
  - cce(y_true, y_pred) : fungsi ini akan menghitung categorical cross-entropy, yang digunakan untuk klasifikasi multiclass yang berpasangan dengan softmax. Lalu, untuk mencegah terjadi overflow, nilai dibatasi menjadi (1e-15, 1 - 1e-15). CCE dapat dirumuskan secara matematis sebagai berikut
    $$L_{\text{CCE}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{c} y_{i,j}\log(\hat{y}_{i,j})$$

- Method turunan, yang digunakan untuk backpropagation:
  - mse_deriv(y_true, y_pred) : fungsi akan menghitung turunan dari MSE dan mengembalikan nilai tersebut. Turunan MSE dapat dirumuskan sebagai berikut $-\ (\frac{2}{n})\ *\ (y_{true} -\ y_{pred})$.

- bce_deriv (y_true, y_pred) : fungsi akan menghitung turunan parsial dari BCE dan mengembalikan nilai tersebut. Turunan dari BCE dapat dirumuskan sebagai berikut $-\frac{1}{n}\left(\frac{y_{true}}{y_{pred}} - \frac{1-y_{true}}{1-y_{pred}}\right)$.
- cce_deriv (y_true, y_pred) : fungsi akan menghitung turunan parsial dari CCE dan mengembalikan nilai tersebut. Turunan dari CCE dapat dirumuskan sebagai berikut $-\frac{1}{n}\left(\frac{y_{true}}{y_{pred}}\right)$.

c. Class RMSNorm

Kelas ini berguna untuk mengimplementasikan RMS Normalization pada hidden layer agar distribusi nilai layer output tetap terjaga.

- Atribut:
  - eps (epsilon) : nilai kecil untuk mencegah error saat terjadi pembagian dengan nol.
  - gamma : parameter scalling matriks yang bernilai awal 1 dan akan terus terupdate selama proses training.
  - x : cache untuk menyimpan raw input.
  - rms : cache untuk menyimpan nilai RMS.
  - x_norm : cache untuk menyimpan nilai input yang sudah dinormalisasi pada saat fase forward.
  - d_gamma : menyimpan nilai gradien gamma.
  - m_gamma : meyimpan nilai rata-rata pergerakan dari gradien parameter gamma
  - v_gamma : menyimpan nilai rata-rata pergerakan dari kuadrat gradien parameter gamma
- Method:
  - __init__(size, eps) : konstruktor untuk membuat matriks parameter dan atribut cache sesuai dengan ukuran dimensi output dari layer.
  - forward(x): menghitung nilai RMS dari input sepanjang sumbu fitur, lalu membagi input dengan RMS tersebut, dan mengalikannya dengan parameter skala gamma.
  - backward(d_out): Menerima gradien dari proses sebelumnya, lalu menghitung turunan parsial terhadap gamma dan turunan parsial terhadap input asli menggunakan prinsip chain rule.

d. Class Layer

Kelas ini digunakan untuk merepresentasikan satu layer pada arsitektur NN.

- Atribut:
  - Input_size : mendefinisikan jumlah dimensi input layer.
  - output_size : mendefinisikan jumlah dimensi output layer.
  - activation_name, act_func, act_deriv : Menyimpan referensi ke fungsi aktivasi dan turunannya yang diambil dari pemetaan self.activations.
  - use_rmsnorm : atribut yang menentukan apakah RMSNorm digunakan atau tidak.

    - rmsnorm : menyimpan objek RMSNorm untuk digunakan pada layer jika rmsnorm diaktifkan.
    - weights : menyimpan matrix bobot, diinisialisasi berdasarkan method yang digunakan (Uniform, Normal, Xavier, He, atau Zero).
    - bias : menyimpan vektor bias.
    - inputs : menyimpan nilai input.
    - z : menyimpan hasil dot product input dengan bobot serta dijumlahkan dengan bias.
    - z_pre_act : menyimpan nilai z setelah normalisasi, tapi sebelum aktivasi.
    - d_weights : menyimpan hasil kalkulasi gradien bobot.
    - d_bias : menyimpan hasil kalkulasi gradien bias.
    - m_w : memori untuk menyimpan history momentum bobot.
    - m_b : memori untuk menyimpan history momentum bias.
    - v_w : memori untuk menyimpan history varians bobot.
    - v_b : memori untuk menyimpan history varians bias.

- Method :
    - __init__(input_size, output_size, activation, init_method, seed, use_rmsnorm) : konstruktor untuk membangun layer, sesuai dengan parameternya.
    - forward(inputs) : proses komputasi feedforward, yaitu mengalikan matriks input dengan matriks bobot dan menambahkan bias. Kemudian meneruskannya ke RMSNorm, kalau diaktifkan, lalu memasukkannya ke dalam fungsi aktivasi.
    - backward(d_out) : proses komputasi backpropagation, yaitu mengalikan gradien layer setelahnya dengan turunan fungsi aktivasi (serta turunan RMSNorm jika diaktifkan), menghitung update gradien untuk matriks bobot dan bias, lalu mengembalikan matriks gradien untuk diteruskan ke layer sebelumnya.

e. Class FFNN

Kelas ini digunakan untuk membangun arsitektur FFNN yang akan digunakan.

- Atribut:
    - layers : list berisi kumpulan layers dari class Layer.
    - loss_func, loss_deriv, loss_name: referensi fungsi loss dari class Loss yang dipilih saat inisialisasi model.
- Method
    - __init__(loss) : konstruktor kelas FFNN untuk inisialisasi model FFNN dan nentuin fungsi loss yang digunakan selama proses training.
    - add(layer) : menerima objek Layer dan menambahkannya ke list self.layers untuk menambah dimensi arsitektur model.

- fit(X, y, batch_size, epochs, learning_rate, verbose, optimizer, l1, l2, validation_data) : proses komputasi untuk training model. Selama epochs yang ditentukan, fungsi akan terus mengacak dataset dan membaginya ke dalam mini-batch sebesar batch_size, dan melakukan fase forward propagation untuk memprediksi hasil serta menghitung lossnya. Lalu, fungsi menjalankan backward propagation di setiap layer untuk menghitung gradien loss, yang selanjutnya digunakan oleh optimizer untuk update nilai bobot dan bias untuk meningkatkan akurasi model di epoch berikutnya.
- plot_weight_distribution(layer_indices) : method untuk menampilkan distribusi bobot.
- plot_gradient_distribution(self, layer_indices) : method untuk menampilkan distribusi gradien.
- save(filepath) : method untuk menyimpan model.
- load(filepath) : method untuk meload model.

2. Forward Propagation

Pada FFNN, forward propagation adalah proses yang terus bergerak maju untuk tiap layer, tanpa ada loop kebelakang, hingga menghasilkan nilai prediksi.

```
    def predict(self, X):
        out = X
        for layer in self.layers:
            out = layer.forward(out)
        return out
```

Fungsi ini berguna untuk menjalankan matriks input melalui semua layer secara berurutan hingga ke layer terakhir.

```
    def forward(self, inputs):
        self.inputs = inputs
        self.z = np.dot(inputs, self.weights) + self.bias
```

Pada tahap ini matriks input dioperasikan secara dot product dengan matriks bobot, lalu kemudian ditambah dengan vektor bias. Nilai input asli dan nilai z ini disimpan di dalam atribut layer karena nantinya akan dipakai untuk menghitung chain rule saat proses backward propagation.

```
        if self.use_rmsnorm:
            self.z_pre_act = self.rmsnorm.forward(self.z)
        else:
```

```
            self.z_pre_act = self.z
```

Kalau pada layer tersebut menggunakan RMSNorm, nilai z akan melalui method forward milik kelas RMSNorm untuk dinormalisasi sebelum diaktivasi..

```
    def forward(self, x):
        self.x = x
        self.rms = np.sqrt(np.mean(x**2, axis=1, keepdims=True)
+ self.eps)
        self.x_norm = x / self.rms
        return self.gamma * self.x_norm
```

Di dalam RMSNorm, dihitung RMS, lalu matriks input dibagi dengan nilai tersebut, dan dikalikan dengan parameter trainable gamma agar angkanya stabil dan tidak exploding.

```
        return self.act_func(self.z_pre_act)
```

Lalu, nilai sebelum aktivasinya diinput ke dalam fungsi aktivasi. Output aktivasi tersebut kemudian akan diteruskan ke layer depannya.

3. Backward Propagation

Pada FFNN, Backward propagation merupakan proses menghitung tingkat kemiringan error (gradien) secara mundur, untuk melihat pengaruh weight dan bias ke tingkat error untuk diperbaiki nilainya.

```
                if self.loss_name == "cce" and
self.layers[-1].activation_name == "softmax":
                    d_out = (out - y_batch) / X_batch.shape[0]
                else:
                    d_out = self.loss_deriv(y_batch, out)
```

Tahap ini digunakan untuk menghitung turunan pertama dari fungsi Loss, untuk mengetahui tingkat error sebelum menjalani backward propagation. Jika model menggunakan kombinasi CCE dan Softmax, nilai gradien awalnya langsung dihitung menggunakan persamaan (out - y_batch) / N, yang merupakan cara cepat dibandingkan menghitung turunannya secara terpisah.

```
                for layer in reversed(self.layers):
                    d_out = layer.backward(d_out)
```

Setelah nilai gradien error pertama diperoleh, gradien tersebut akan berjalan mundur melewati setiap layer menggunakan looping terbalik. Setiap layer akan memproses gradien yang masuk, menghitung gradien bobotnya sendiri, lalu merubah nilai gradien tersebut untuk diberikan ke layer sebelumnya.

Ketika nilai gradient masuk ke dalam suatu Layer, proses chain rule terjadi secara berurutan dengan arah terbalik

```
    def backward(self, d_out):
        if self.activation_name != "softmax":
            d_z_pre_act = d_out *
self.act_deriv(self.z_pre_act)
        else:
            d_z_pre_act = d_out
```

Gradien yang masuk dikalikan dengan turunan dari fungsi aktivasi layer tersebut. Bagian ini berfungsi untuk menentukan seberapa besar error yang boleh lewat berdasarkan kondisi neuron saat forward.

```
        if self.use_rmsnorm:
            d_z = self.rmsnorm.backward(d_z_pre_act)
        else:
            d_z = d_z_pre_act
```

```
    def backward(self, d_out):    # RMSNorm
        N, D = d_out.shape
        self.d_gamma = np.sum(d_out * self.x_norm, axis=0,
keepdims=True)

        d_x_norm = d_out * self.gamma

        d_rms = np.sum(d_x_norm * self.x * (-1.0 /
(self.rms**2)), axis=1, keepdims=True)
        d_x = (d_x_norm / self.rms) + (d_rms * self.x / (D *
self.rms))

        return d_x
```

Jika layer menggunakan RMSNorm, gradien akan dilewatkan dulu ke fungsi backward milik RMSNorm untuk menghitung turunan parameter gamma sekaligus menyesuaikan gradien terhadap nilai RMS.

```
        self.d_weights = np.dot(self.inputs.T, d_z)
        self.d_bias = np.sum(d_z, axis=0, keepdims=True)
```

Pada tahap ini, gradien untuk matriks bobot dihitung menggunakan perkalian dot product antara matriks input yang sudah di-transpose dengan gradien saat ini. Sedangkan gradien bias didapat dengan menjumlahkan gradien secara vertikal.

```
        d_inputs = np.dot(d_z, self.weights.T)
        return d_inputs
```

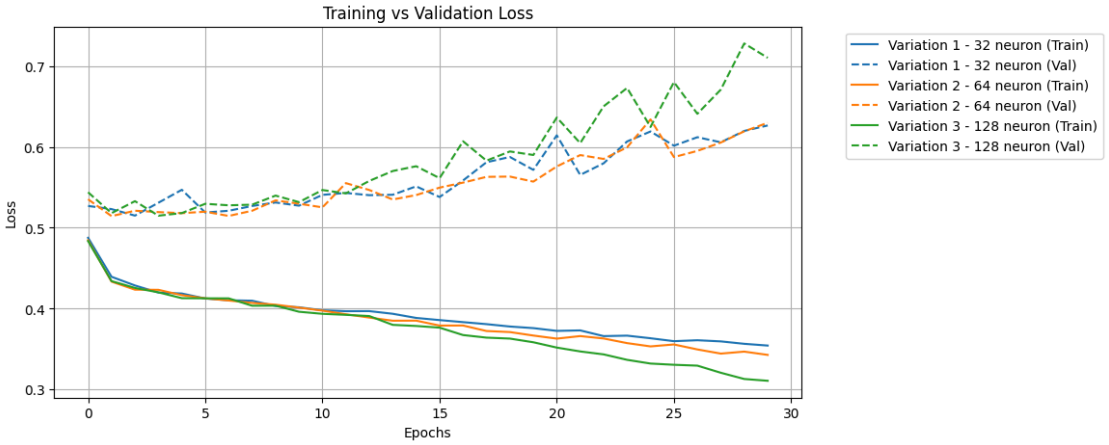
Selanjutnya, gradien saat ini dikalikan dengan matriks bobot layer ini yang di-transpose. Hasilnya dikembalikan dan dipakai oleh looping di FFNN sebagai gradien yang baru untuk layer berikutnya.

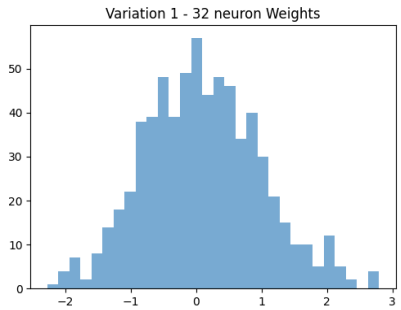
Setelah backward propagation selesai menyusuri semua layer, proses selanjutnya adalah nilai weights dan bias di setiap layer menggunakan gradien yang baru saja dihitung.

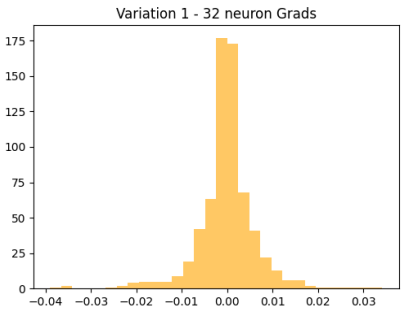
Hasil Pengujian

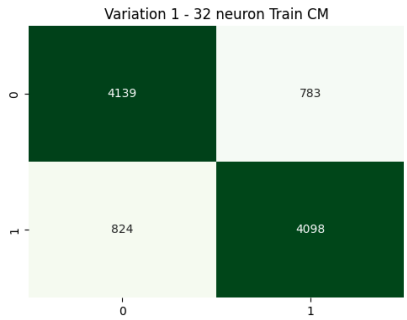
1. Pengaruh depth dan width.

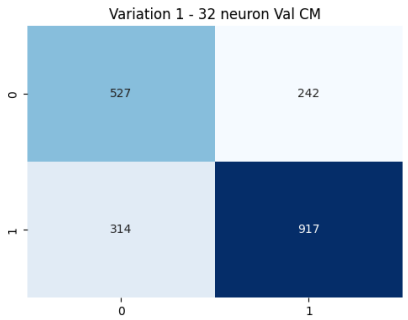
a. Pengujian Variasi Width



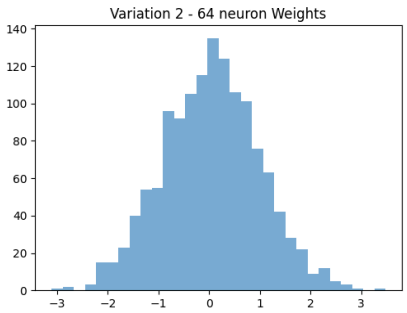


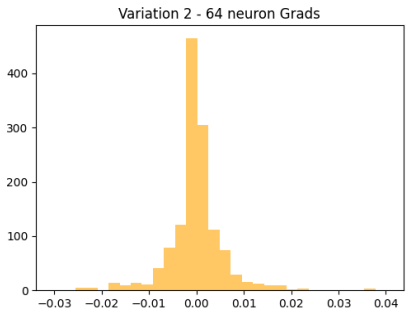


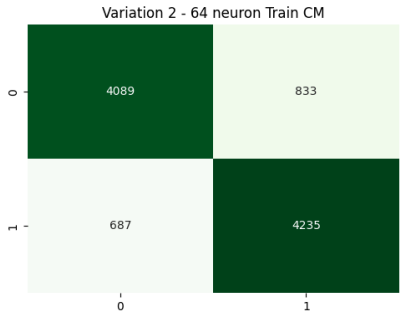


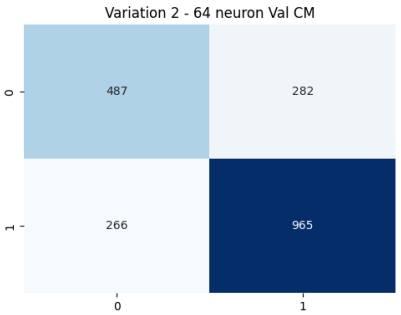



```
EXPERIMENT: Variation 1 - 32 neuron
ACCURACY | Train: 0.8368 | Validation: 0.7220

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.63      0.69      0.65       769
           1       0.79      0.74      0.77      1231

    accuracy                           0.72      2000
   macro avg       0.71      0.72      0.71      2000
weighted avg       0.73      0.72      0.72      2000
```

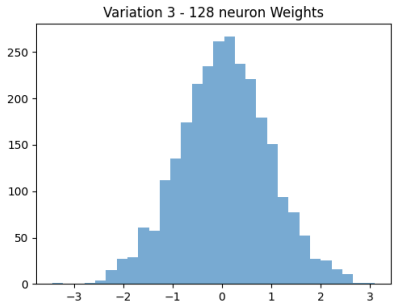


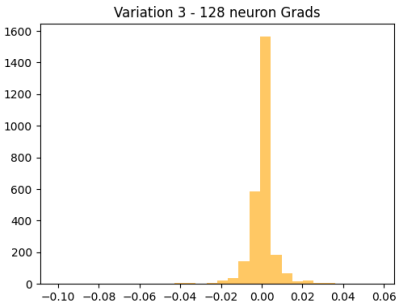


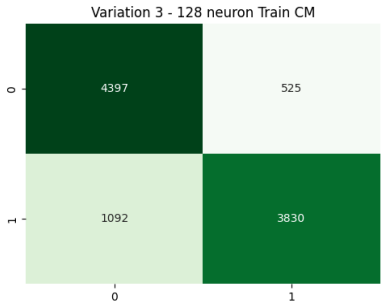


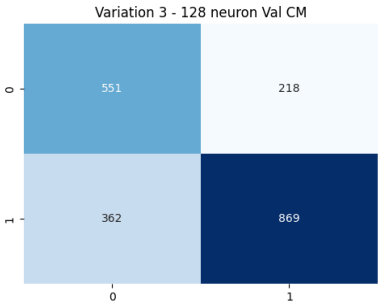



```
EXPERIMENT: Variation 2 - 64 neuron
ACCURACY | Train: 0.8456 | Validation: 0.7260

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.65      0.63      0.64       769
           1       0.77      0.78      0.78      1231

    accuracy                           0.73      2000
   macro avg       0.71      0.71      0.71      2000
weighted avg       0.72      0.73      0.73      2000
```







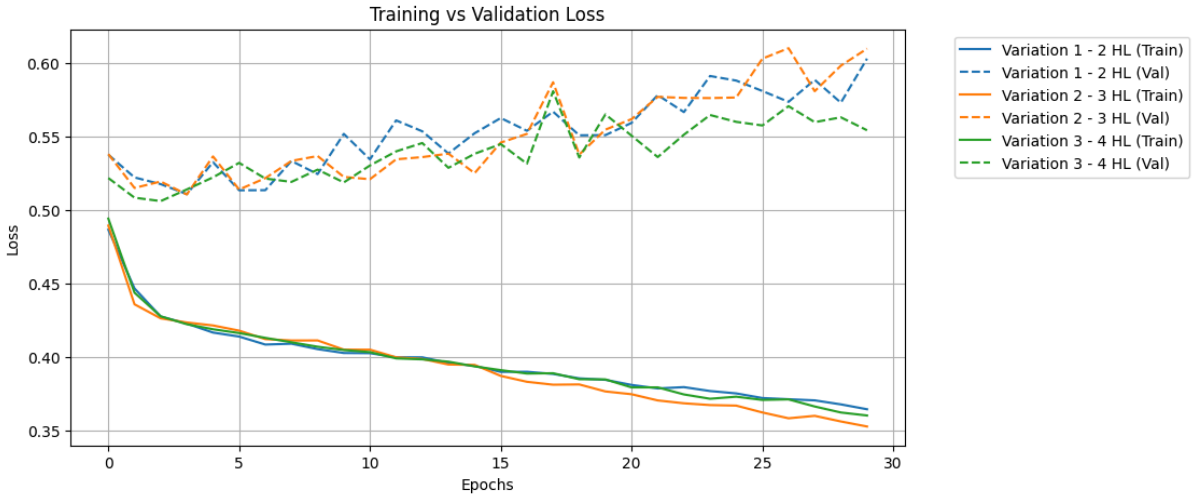


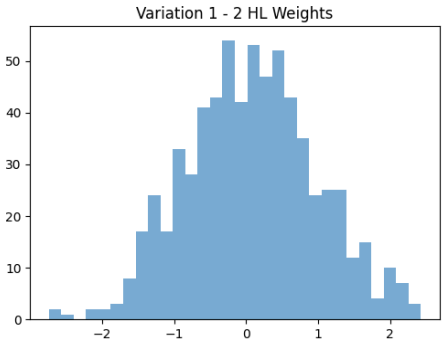
```
EXPERIMENT: Variation 3 - 128 neuron
ACCURACY | Train: 0.8357 | Validation: 0.7100

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.60      0.72      0.66       769
           1       0.80      0.71      0.75      1231

    accuracy                           0.71      2000
   macro avg       0.70      0.71      0.70      2000
weighted avg       0.72      0.71      0.71      2000
```

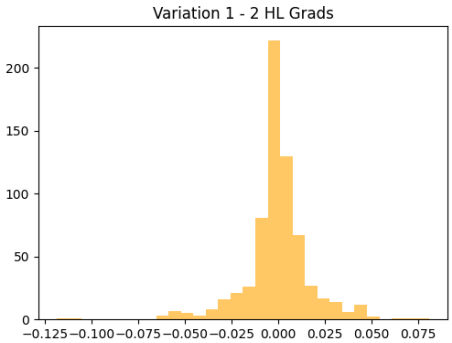
Pada pengujian variasi Width ini diujikan 3 arsitektur dengan jumlah hidden layer pertama sejumlah 32, 64, dan 128. Dari hasil percobaan didapatkan bahwa model dengan HL 0 berjumlah 64 memberikan dengan Val Accuracy tertinggi bernilai 0.726. Hal ini menunjukkan bahwa semakin banyak parameter dalam satu layer meningkatkan kemampuan model untuk meniru pola dataset dengan lebih baik, namun pada nilai parameter yang terlalu tinggi dapat menyebabkan overfitting dimana model malah gagal melakukan generalisasi dan meniru penuh dataset. Terlihat pada parameter tertinggi 128 neuron menghasilkan Training Accuracy tertinggi di 0.835 namun memiliki Val Accuracy terendah di 0.710 yang mengindikasikan model gagal melakukan generalisasi akibat ovefitting terhadap data training.

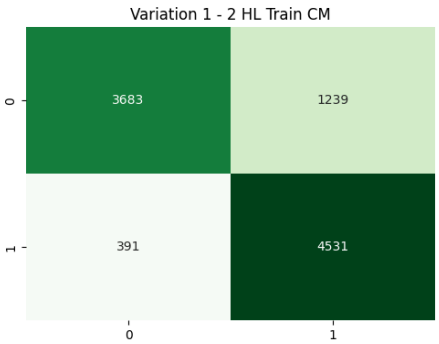
Pada percobaan ini terlihat juga distribusi weight cukup baik berdistribusi normal dan untuk gradient tidak ada ledakan ataupun indikasi vanishing.

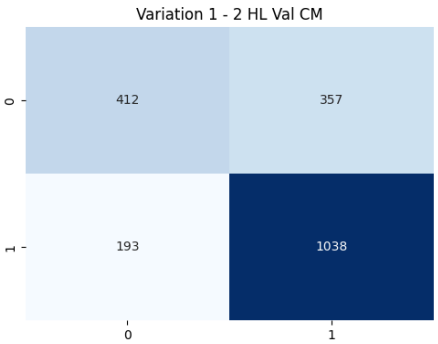
b. Pengujian Depth



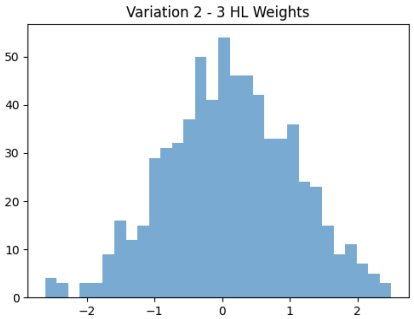


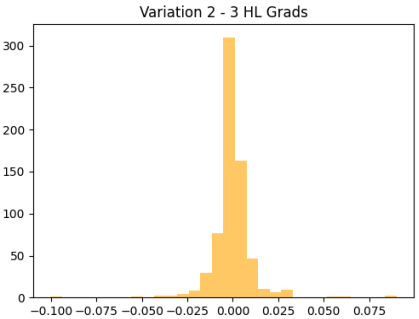


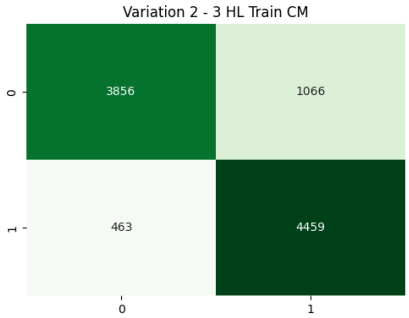


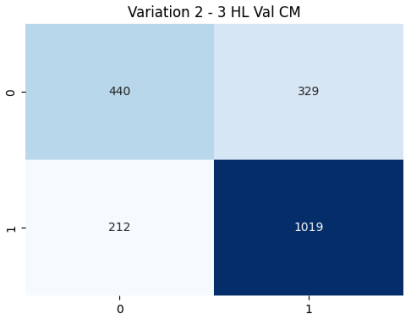



```
EXPERIMENT: Variation 1 - 2 HL
ACCURACY | Train: 0.8344 | Validation: 0.7250

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.68      0.54      0.60       769
           1       0.74      0.84      0.79      1231

    accuracy                           0.72      2000
   macro avg       0.71      0.69      0.70      2000
weighted avg       0.72      0.72      0.72      2000
```

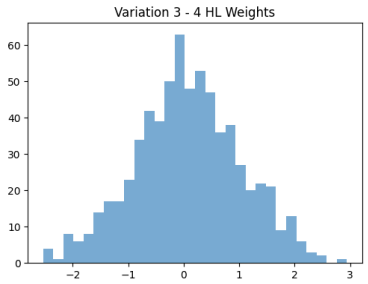


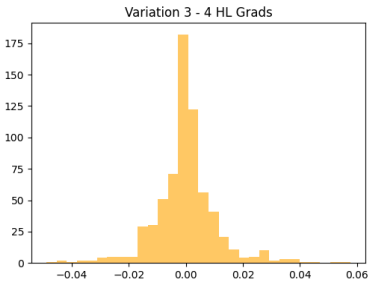


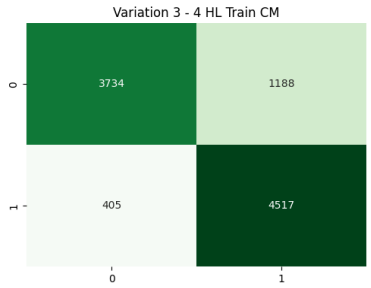


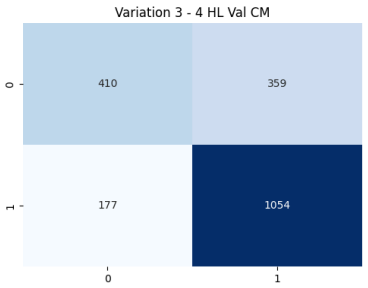



```
EXPERIMENT: Variation 2 - 3 HL
ACCURACY | Train: 0.8447 | Validation: 0.7295

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.67      0.57      0.62       769
           1       0.76      0.83      0.79      1231

    accuracy                           0.73      2000
   macro avg       0.72      0.70      0.70      2000
weighted avg       0.72      0.73      0.72      2000
```





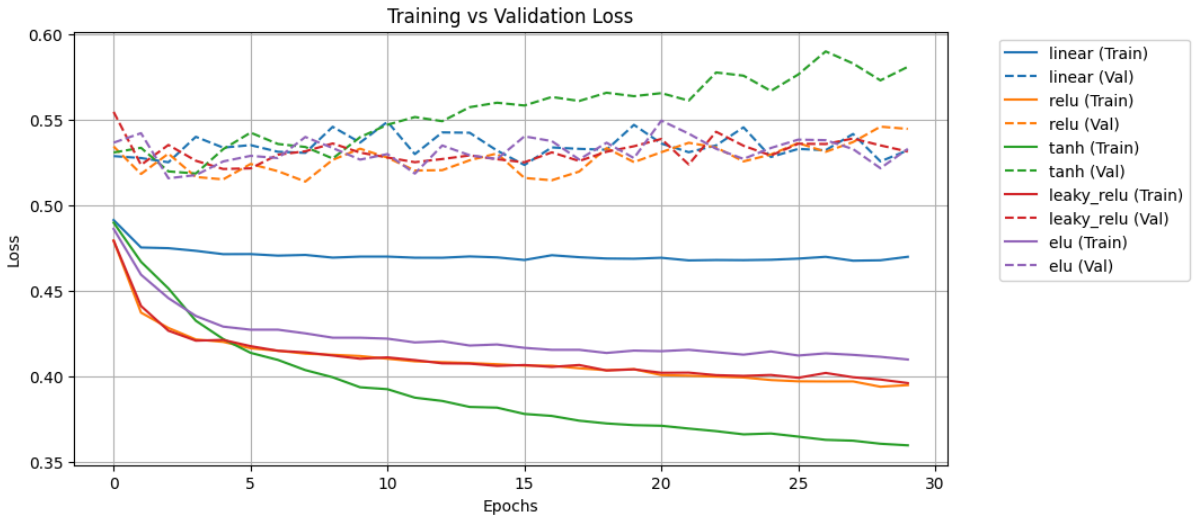


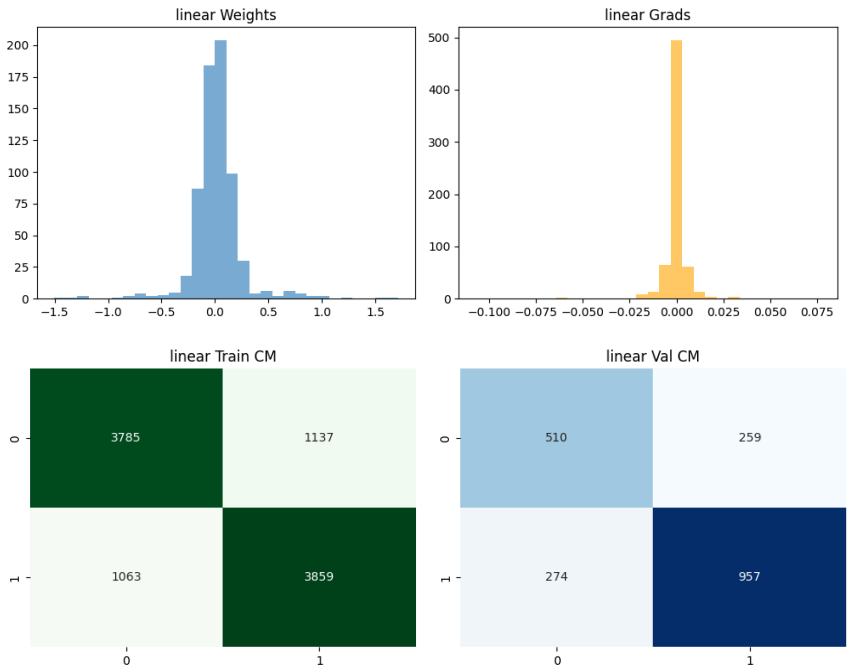



```
EXPERIMENT: Variation 3 - 4 HL
ACCURACY | Train: 0.8382 | Validation: 0.7320

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.70      0.53      0.60       769
           1       0.75      0.86      0.80      1231

    accuracy                           0.73      2000
   macro avg       0.72      0.69      0.70      2000
weighted avg       0.73      0.73      0.72      2000
```

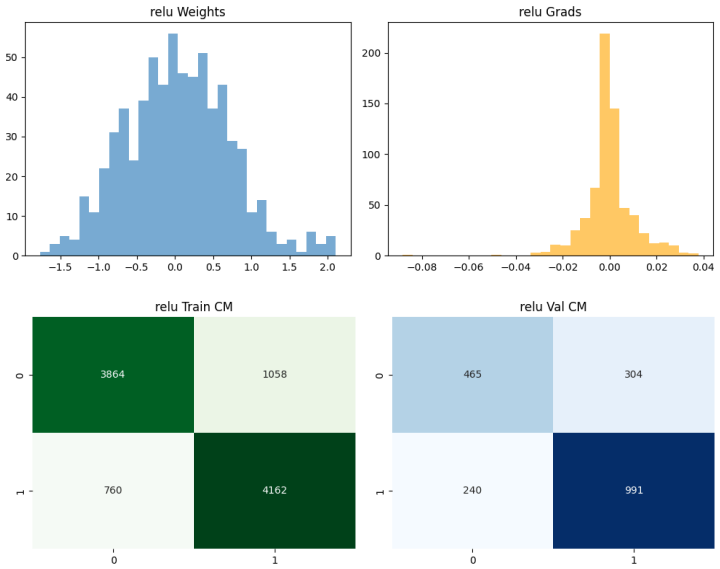
Pada percobaan ini melakukan variasi kedalaman hidden layer dengan variasi 2, 3, dan 4 HL. Dari hasil percobaan didapatkan nilai Val Accuracy terbaik didapat oleh 4 HL. Namun perlu diwaspadai kemungkinan bahwa apabila ditambahkan lagi HL akan menyebabkan overfitting pada model akibat kompleksitas yang terlalu tinggi pada desain arsitektur. Terdapat juga potensi masalah degradation akibat vanishing gradient antar layer. Masih sama bahwa pada percobaan ini juga nilai beban juga terdistribusi dengan baik, serta nilai gradient tidak ada indikasi vanishing atau ledakan.

2. Pengaruh fungsi aktivasi.

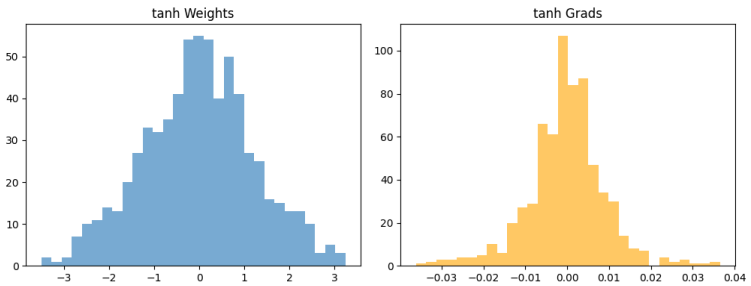


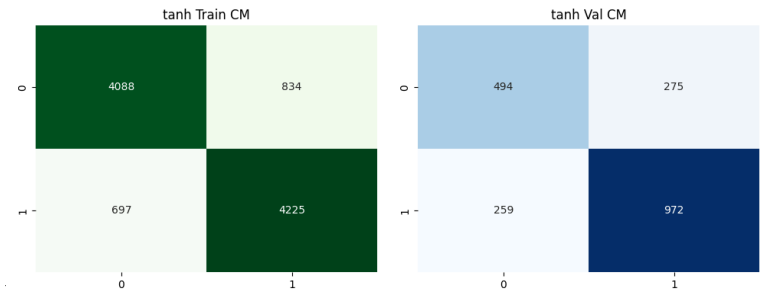


```
EXPERIMENT: linear
ACCURACY | Train: 0.7765 | Validation: 0.7335

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.65      0.66      0.66       769
           1       0.79      0.78      0.78      1231

    accuracy                           0.73      2000
   macro avg       0.72      0.72      0.72      2000
weighted avg       0.73      0.73      0.73      2000
```

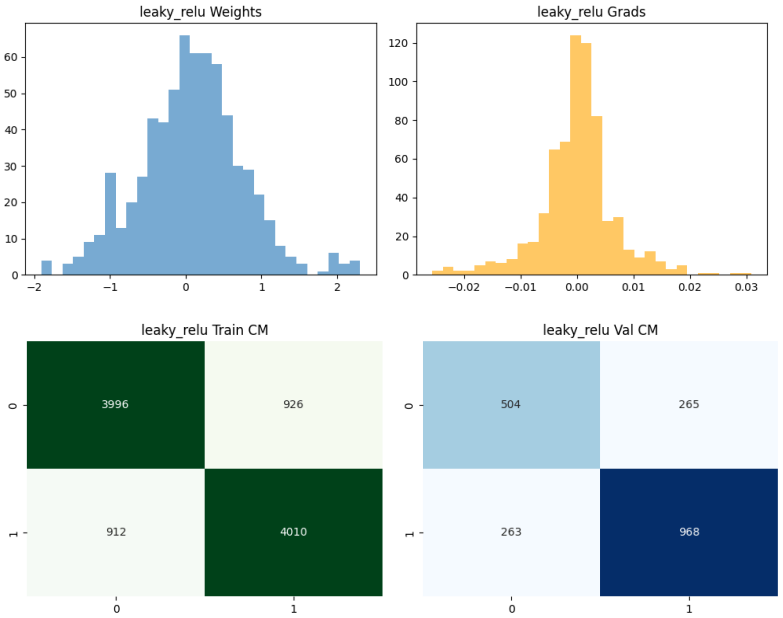



```
EXPERIMENT: relu
ACCURACY | Train: 0.8153 | Validation: 0.7280

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.66      0.60      0.63       769
           1       0.77      0.81      0.78      1231

    accuracy                           0.73      2000
   macro avg       0.71      0.70      0.71      2000
weighted avg       0.72      0.73      0.73      2000
```

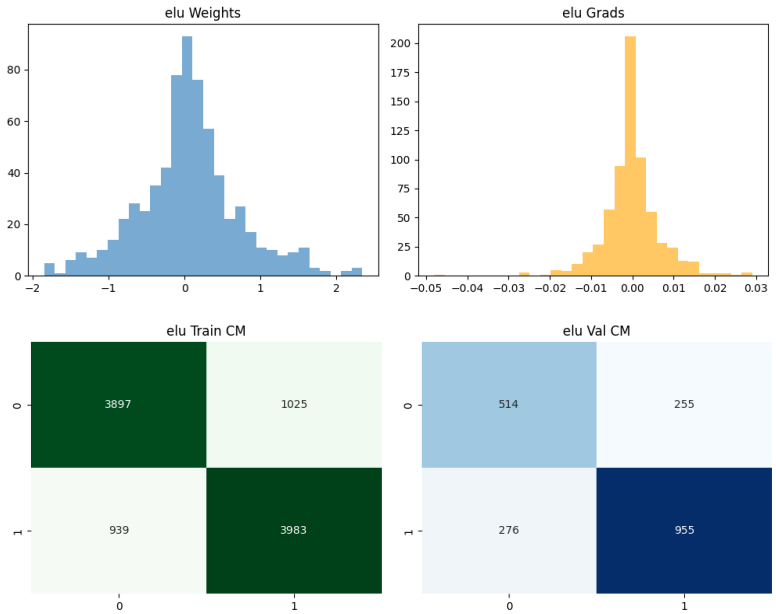





```
EXPERIMENT: tanh
ACCURACY | Train: 0.8445 | Validation: 0.7330

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.66      0.64      0.65       769
           1       0.78      0.79      0.78      1231

    accuracy                           0.73      2000
   macro avg       0.72      0.72      0.72      2000
weighted avg       0.73      0.73      0.73      2000
```




```
EXPERIMENT: leaky_relu
ACCURACY | Train: 0.8133 | Validation: 0.7360

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.66      0.66      0.66       769
           1       0.79      0.79      0.79      1231

    accuracy                           0.74      2000
   macro avg       0.72      0.72      0.72      2000
weighted avg       0.74      0.74      0.74      2000
```

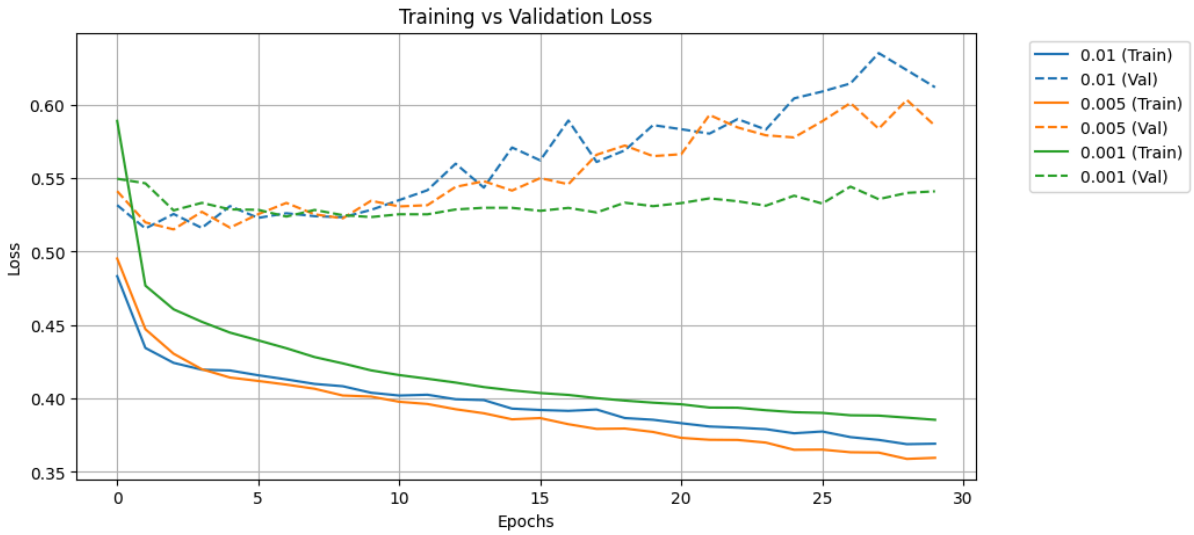



```
EXPERIMENT: elu
ACCURACY | Train: 0.8005 | Validation: 0.7345

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.65      0.67      0.66       769
           1       0.79      0.78      0.78      1231

    accuracy                           0.73      2000
   macro avg       0.72      0.72      0.72      2000
weighted avg       0.74      0.73      0.74      2000
```

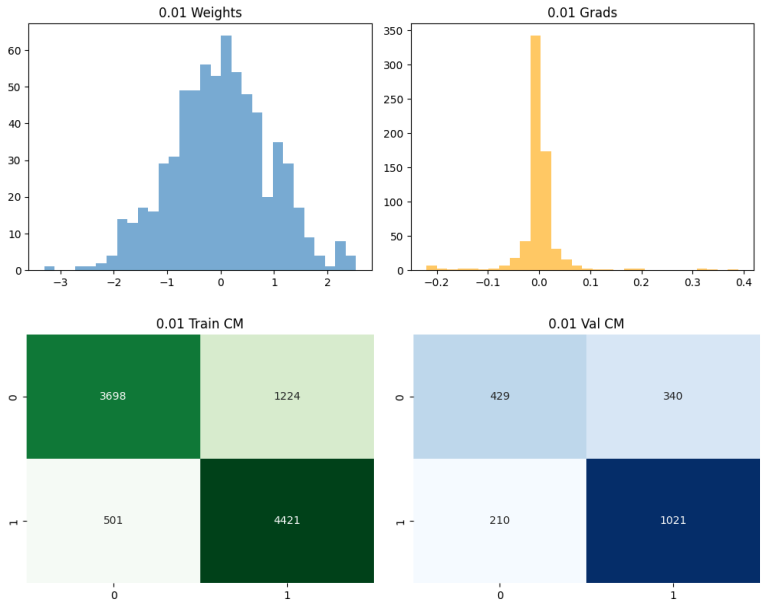
Pada percobaan ini diujikan beberapa activation function pada hidden layer dengan pilihan Linear, ReLU, Tanh, Leaky Relu, dan ELU. Hasil dari percobaan ini menunjukkan bahwa Leaky Relu memberikan hasil Val Accuracy terbaik dengan skor 0.736. ReLU tidak memberikan hasil terbaik kemungkinan akibat dead neuron problem. Dead neuron problem muncul karena pada nilai negatif, nilai keluaran dipaksa 0 dan pada mode backpropagation menyebabkan update bobot yang buta pada rentang negatif. Akibatnya update bobot sulit mendapatkan nilai optimal terhadap keseluruhan parameter Leaky ReLU memberikan nilai dampak perubahan bobot untuk keluaran di rentang negatif sehingga mengatasi masalah ini.

Terdapat insight menarik dimana untuk aktivasi linear bahwa nilai Val Accuracy sudah cukup baik, bahkan mengalahkan ReLU. Menunjukkan kemungkinan bahwa dataset cukup linearly separable.

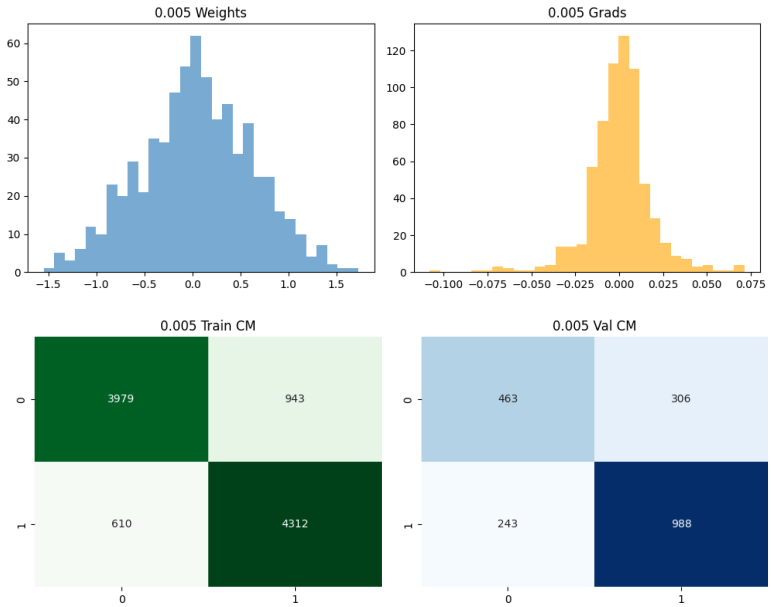
3. Pengaruh learning rate.



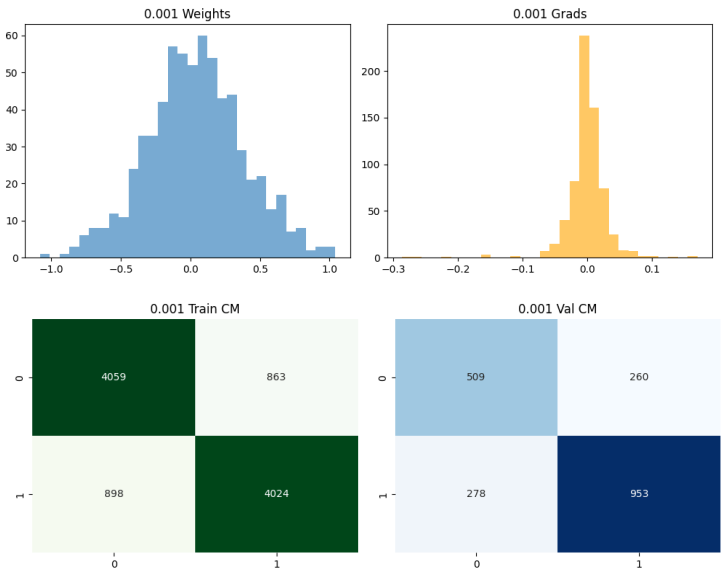
Pada kurva learning rate, terlihat bahwa pada data training,bahwa semakin kecil learning ratenya, maka semakin banyak juga epoch yang diperlukan untuk mencapai nilai loss minimum. Berbanding terbalik dengan model dengan learning rate lebih besar, yang mampu mencapai nilai lebih minimum lebih awal. Namun, karena pada learning rate yang lebih kecil, model akan belajar lebih teliti untuk memperbaiki loss dan akan menjadi lebih general. Pada data validasi, model memiliki performa yang lebih baik, dengan loss yang lebih minimum dibandingkan dengan model dengan learning rate lebih besar. Hal ini juga ditunjukkan oleh learning rate yang lebih besar, yaitu 0.01 dan 0.005, yang menunjukkan kurva tidak stabil dan memiliki loss lebih besar dari model dengan learning rate 0.001. Hal ini karena, model dengan learning rate besar, cenderung terlalu menghafal data training dan juga noise.



```
EXPERIMENT: 0.01
ACCURACY | Train: 0.8248 | Validation: 0.7250

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.67      0.56      0.61       769
           1       0.75      0.83      0.79      1231

    accuracy                           0.72      2000
   macro avg       0.71      0.69      0.70      2000
weighted avg       0.72      0.72      0.72      2000
```



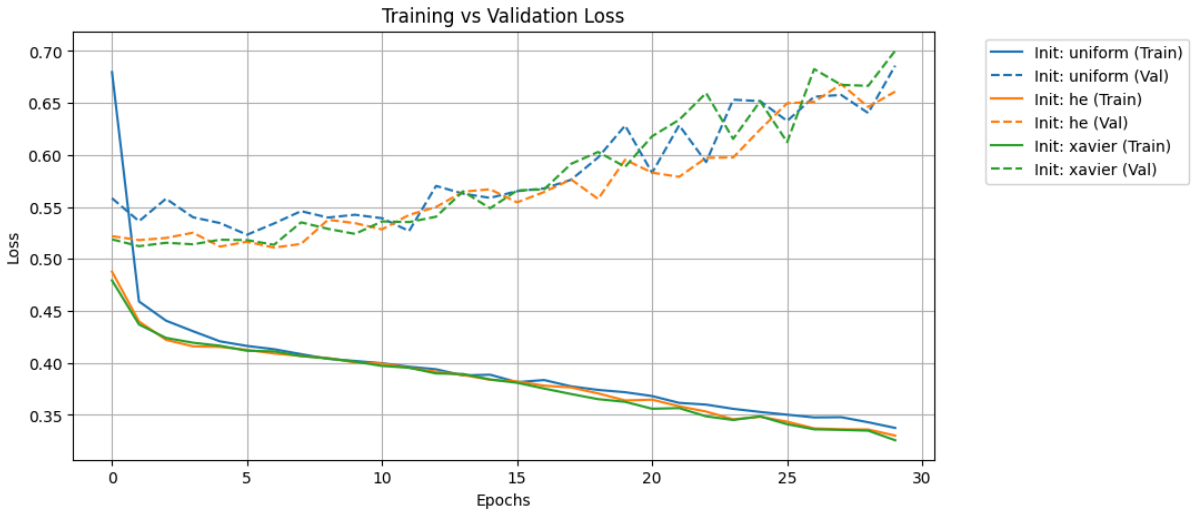

```
EXPERIMENT: 0.005
ACCURACY | Train: 0.8422 | Validation: 0.7255

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.66      0.60      0.63       769
           1       0.76      0.80      0.78      1231

    accuracy                           0.73      2000
   macro avg       0.71      0.70      0.71      2000
weighted avg       0.72      0.73      0.72      2000
```



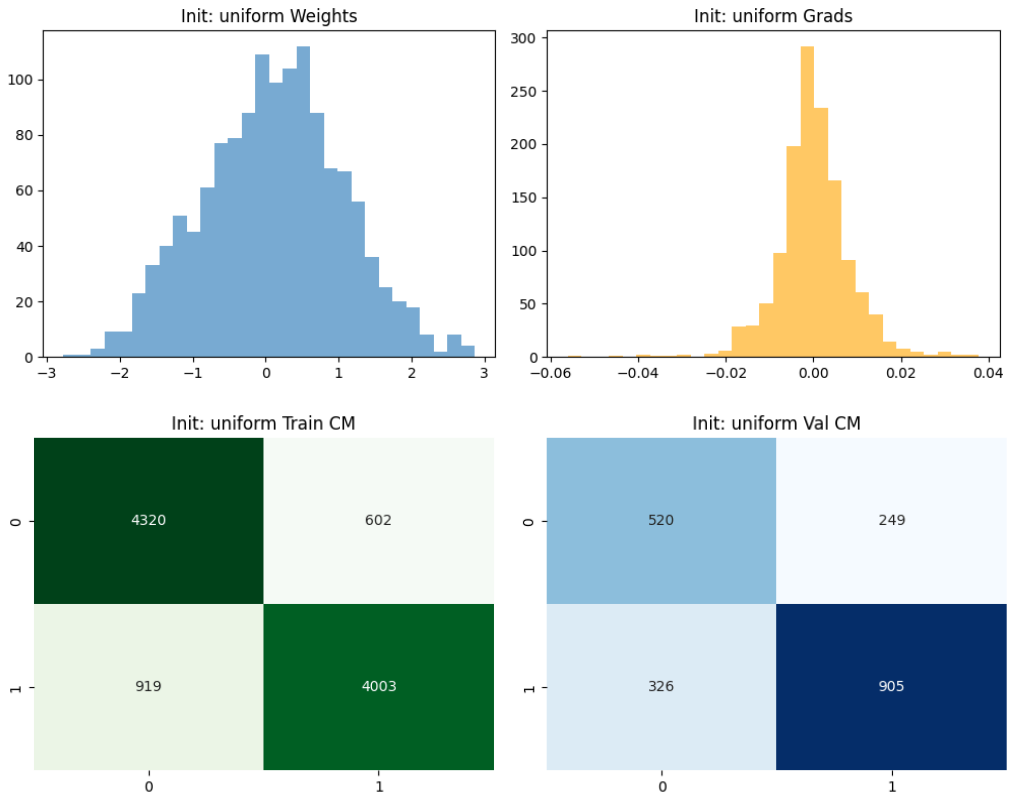

```
EXPERIMENT: 0.001
ACCURACY | Train: 0.8211 | Validation: 0.7310

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.65      0.66      0.65       769
           1       0.79      0.77      0.78      1231

    accuracy                           0.73      2000
   macro avg       0.72      0.72      0.72      2000
weighted avg       0.73      0.73      0.73      2000
```

Pada metrik klasifikasi, terlihat bahwa model dengan learning rate yang lebih kecil mampu membuat generalisasi yang lebih baik. Terlihat dari recall pada kedua class, dengan tebakan kelas mayoritas (Kelas 1) dan kelas minoritas (Kelas 0) memiliki nilai yang cukup tinggi dan seimbang. Hal ini karena, dengan learning rate yang lebih rendah, model mempelajari tiap fitur dari data dengan lebih teliti, sehingga mampu memberikan valuasi untuk tiap fitur dengan lebih baik. Hal ini berbeda dengan model dengan learning rate yang lebih besar yang cenderung mengalami bias yang cukup kuat, sehingga cenderung menebak kelas mayoritas. Pada model dengan learning rate besar, model cenderung ingin menekan nilai loss secara cepat, sehingga model mengambil keputusan untuk cenderung memilih kelas mayoritas yang secara matematis memiliki probabilitas benar lebih tinggi, yang dapat mengorbankan tebakan kelas minoritas.

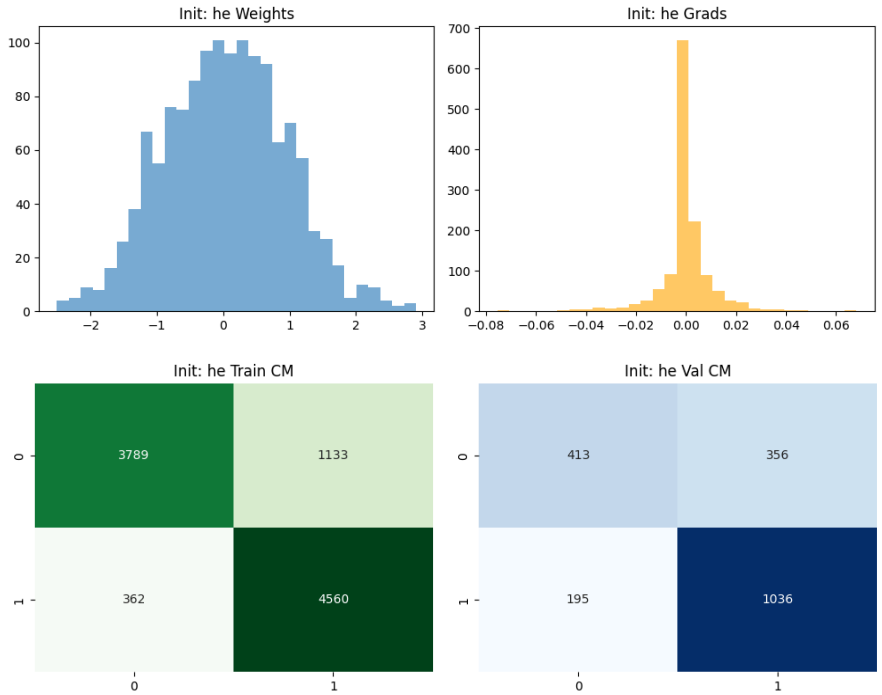
4. Pengaruh inisialisasi bobot.



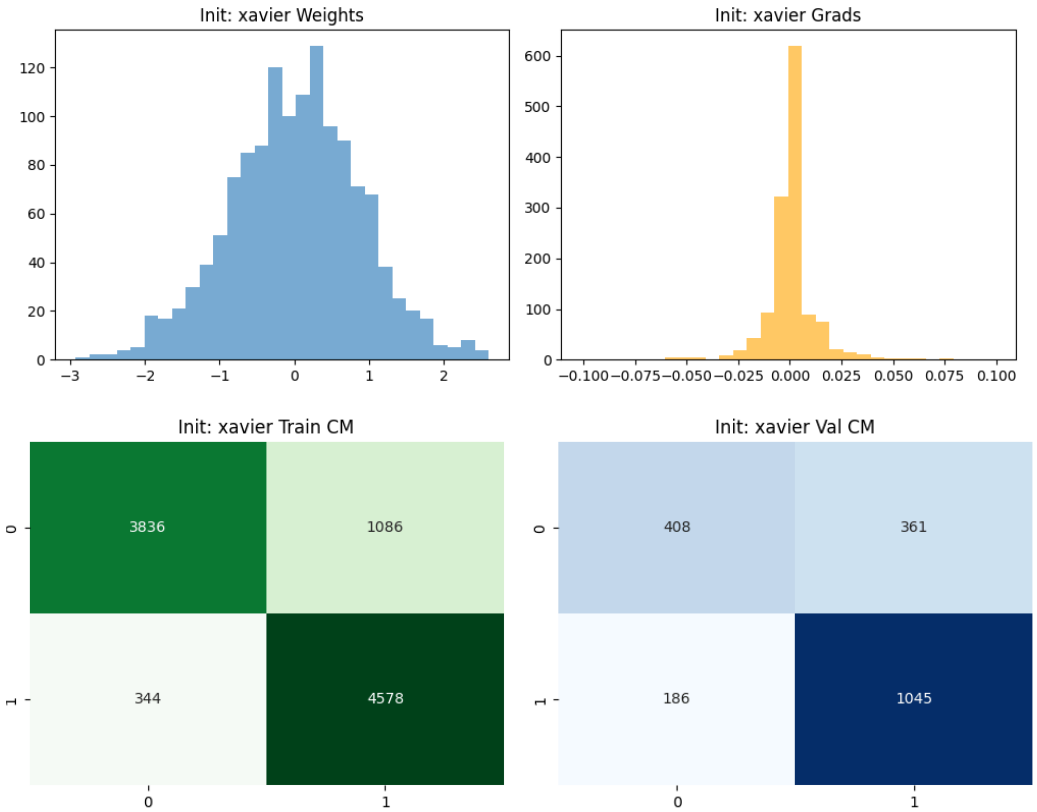
Pada kurva training dan validation loss, model dengan inisialisasi uniform menunjukkan tingkat loss awal yang lebih tinggi serta penurunan yang lambat dan tidak mulus pada epoch awal karena bobot disebar secara acak tanpa memperhitungkan jumlah neuron, sehingga gradien tidak optimal. Sebaliknya, inisialisasi he dan xavier menunjukkan penurunan loss yang jauh lebih cepat dan mulus sejak epoch pertama. Hal ini terjadi karena kedua metode tersebut dirancang secara matematis untuk menjaga stabilitas varians aktivasi dari satu layer ke layer berikutnya, sehingga optimizer dapat memperbarui bobot secara langsung dan efektif tanpa hambatan [vanishing gradient](#).



```
EXPERIMENT: Init: uniform
ACCURACY | Train: 0.8455 | Validation: 0.7125

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.61      0.68      0.64       769
           1       0.78      0.74      0.76      1231

    accuracy                           0.71      2000
   macro avg       0.70      0.71      0.70      2000
weighted avg       0.72      0.71      0.71      2000
```




```
EXPERIMENT: Init: he
ACCURACY | Train: 0.8481 | Validation: 0.7245

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.68      0.54      0.60       769
           1       0.74      0.84      0.79      1231

    accuracy                           0.72      2000
   macro avg       0.71      0.69      0.69      2000
weighted avg       0.72      0.72      0.72      2000
```




```
EXPERIMENT: Init: xavier
ACCURACY | Train: 0.8547 | Validation: 0.7265

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.69      0.53      0.60       769
           1       0.74      0.85      0.79      1231

    accuracy                           0.73      2000
   macro avg       0.72      0.69      0.70      2000
weighted avg       0.72      0.73      0.72      2000
```

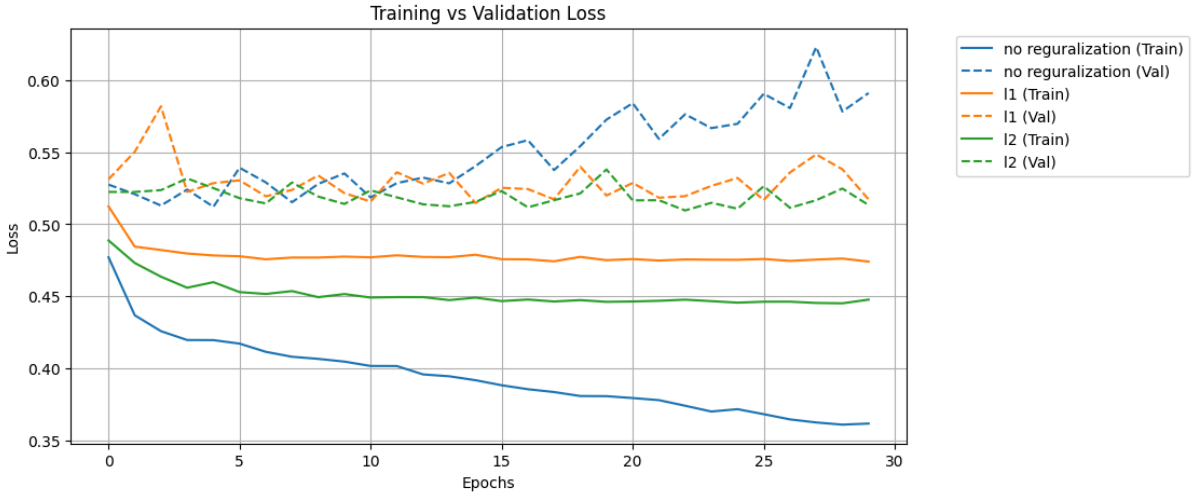
Berdasarkan histogram distribusi bobot, inisialisasi uniform memiliki persebaran seperti segitiga merata. Sedangkan, he, dan xavier distribusi bobotnya membentuk distribusi normal seperti bell. Inisialisasi he memiliki varians penyebaran yang sedikit lebih lebar dibandingkan xavier untuk mengkompensasi sifat fungsi aktivasi ReLU yang mematikan sinyal negatif, sehingga keduanya jauh lebih adaptif dalam menjaga gradien yang baik selama proses backpropagation.

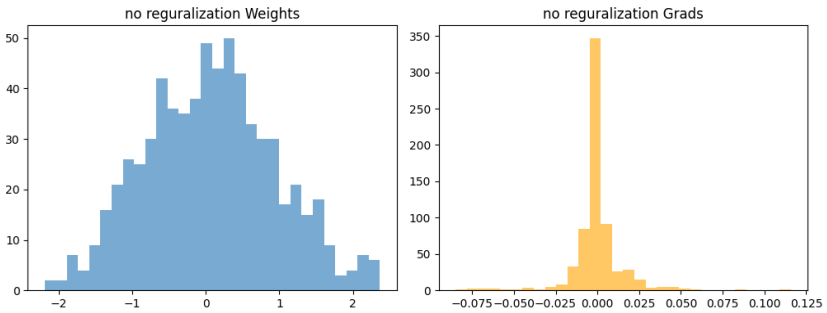
Meskipun pada histogram terlihat bahwa inisialisasi uniform memiliki bentuk yang mirip dengan inisialisasi he dan xavier. Namun, secara matematis, peluang untuk terpilih tiap bobot sama, yang membuat kurva histogram akan lebih merata. Sedangkan, pada distribusi he dan xavier, mengikuti prinsip distribusi normal, yang membuat bentuk kurvanya akan lebih seperti bell. Hal ini juga cukup terlihat dari

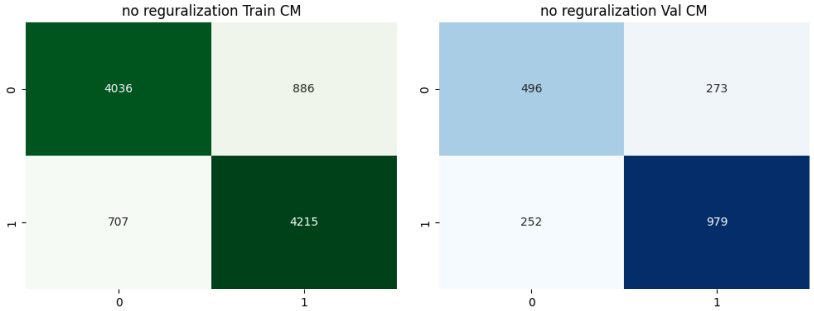
kurva uniform, yang lebih terlihat uniform dibandingkan dengan distribusi he dan xavier.

Hasil metrik klasifikasi menunjukkan bahwa inisialisasi xavier (72.65%) dan he (72.45%) menghasilkan akurasi yang lebih baik dibandingkan uniform (71.25%), tapi memperlihatkan perbedaan recall yang cukup berbeda akibat dataset yang imbalanced. Model dengan inisialisasi uniform terlihat memiliki nilai recall yang lebih seimbang antara Kelas 0 dan Kelas 1, hal tersebut terjadi karena model belum mencapai titik konvergensi yang baik. Sebaliknya, model dengan inisialisasi he dan xavier yang telah belajar dengan lebih baik sehingga dapat menyadari bahwa memprioritaskan prediksi pada kelas mayoritas, kelas 1, adalah cara tercepat untuk meminimalkan loss keseluruhan.

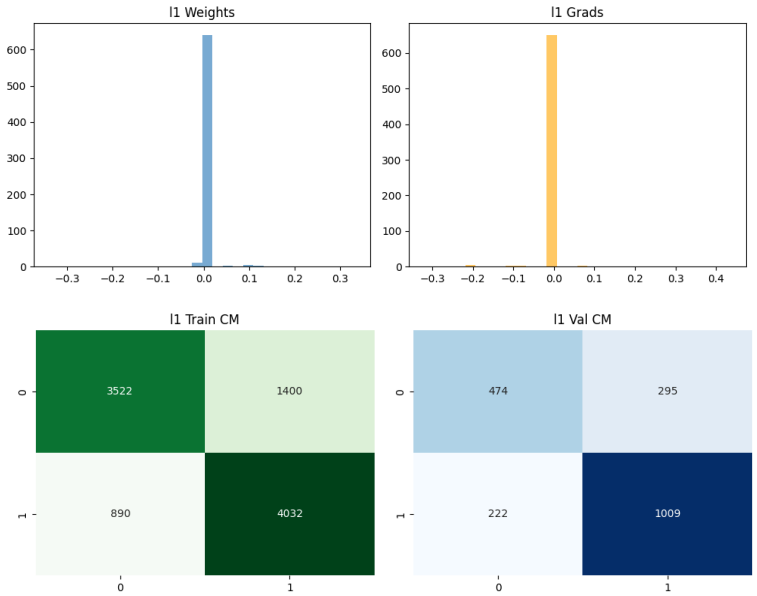
5. Pengaruh regularisasi.



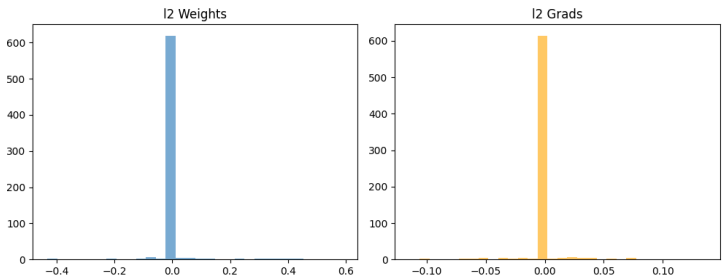


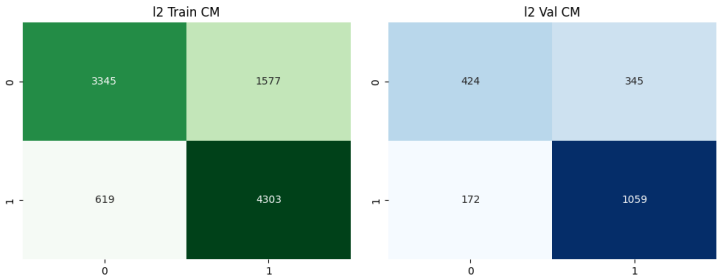


```
EXPERIMENT: no reguralization
ACCURACY | Train: 0.8382 | Validation: 0.7375

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.66      0.64      0.65       769
           1       0.78      0.80      0.79      1231

    accuracy                           0.74      2000
   macro avg       0.72      0.72      0.72      2000
weighted avg       0.74      0.74      0.74      2000
```




```
EXPERIMENT: l1
ACCURACY | Train: 0.7674 | Validation: 0.7415

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.68      0.62      0.65       769
           1       0.77      0.82      0.80      1231

    accuracy                           0.74      2000
   macro avg       0.73      0.72      0.72      2000
weighted avg       0.74      0.74      0.74      2000
```

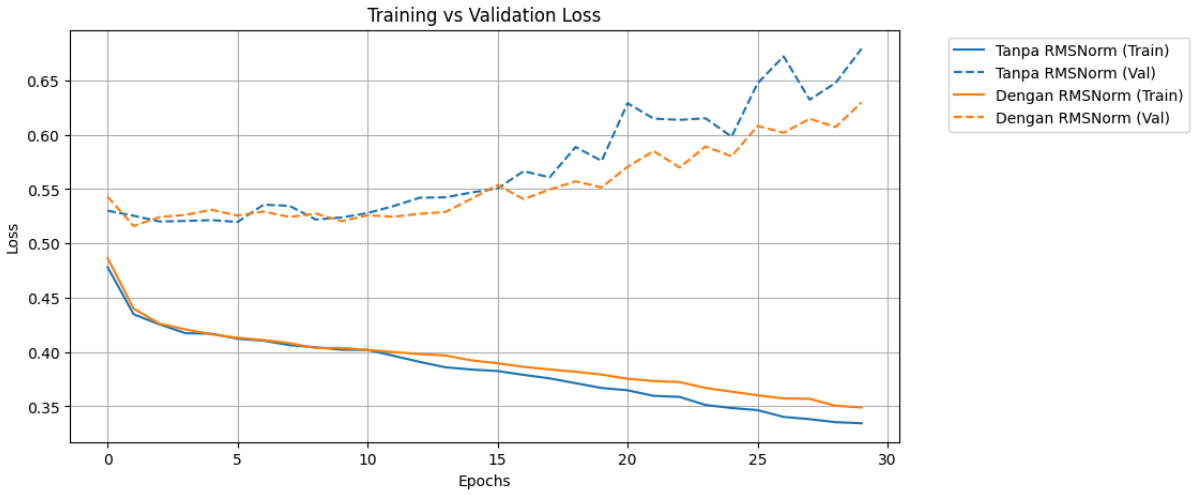




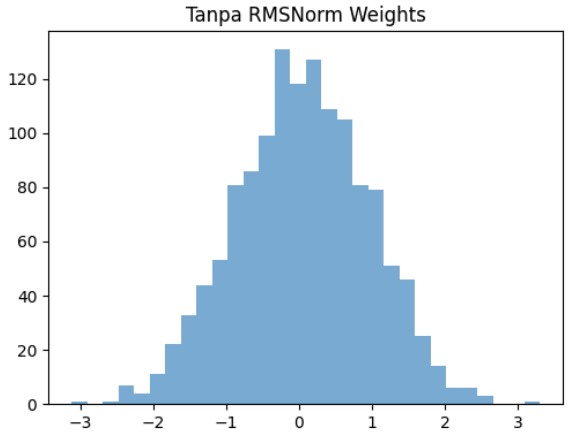

```
EXPERIMENT: l2
ACCURACY | Train: 0.7769 | Validation: 0.7415

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.71      0.55      0.62       769
           1       0.75      0.86      0.80      1231

    accuracy                           0.74      2000
   macro avg       0.73      0.71      0.71      2000
weighted avg       0.74      0.74      0.73      2000
```

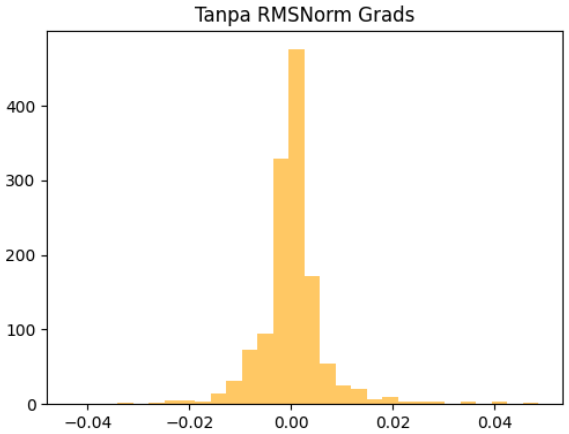
Regularisasi adalah metode untuk menyederhanakan model dengan memaksa nilai nilai kecil agar dibulatkan ke nol. Hal ini menyebabkan model lebih ringan untuk dimuat atau dioperasikan. Namun metode ini tentu memiliki trade off penurunan kemampuan akurasi dan generalisasi. Regularisasi L1 akan menambahkan penalti pada |w| sehingga medorong banyak weight menjadi 0. semenara L2 melakukan penalti pada W^2, mengecilkan semua weight dengan lebih smooth sehingga distribusi weight masih tergolong normal dengan spike di nilai 0. Hal ini terlihat pada weight distribution dimana tanpa Regularisasi menghasilkan distribusi normal, negan L1 Lasso didapatkan spike tajam di nilai 0, dan dengan L2 memberikan sedikit nilai dengan distribusi normal meski spike di 0. Secara performa juga didapatkan bahwa untuk tanpa regularisasi didapatkan training accuracy terbesar namun val nya terendah, sementara nilai val accuracy terbaik dipegang oleh L2.

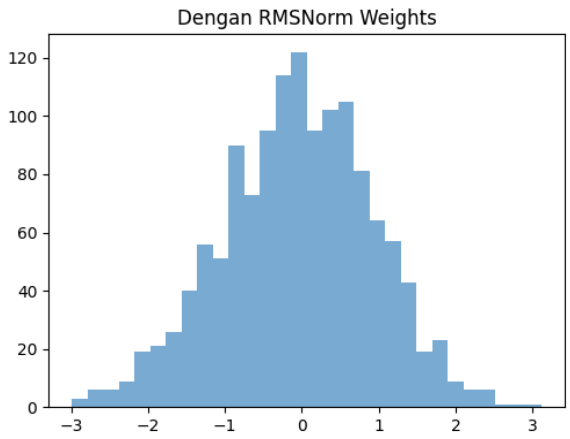
6. Pengaruh normalisasi RMSNorm

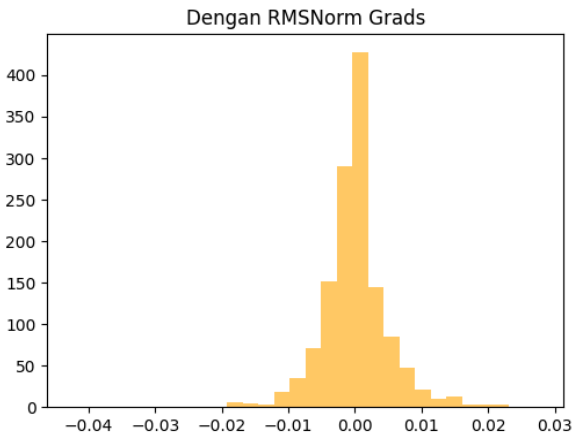


Pada grafik training loss dan validation loss pada kedua model, terdapat perbedaan pola pergerakan konvergensi akibat RMSNorm. Pada model tanpa RMSNorm, kurva loss cenderung mengalami fluktuasi yang lebih tajam dan pergerakan penurunan yang kurang konsisten karena model rentan terhadap skala nilai fitur yang bervariasi pada setiap layer. Ketika RMSNorm diimplementasikan, kurva training loss menunjukkan jalur penurunan yang lebih mulus dan stabil karena RMSNorm menormalkan varians aktivasi sebelum diteruskan ke layer berikutnya. Meskipun demikian, kedua kurva validation loss pada akhirnya menunjukkan plateau, yang menunjukkan bahwa fungsi RMSNorm sangat bagus dalam menjaga stabilitas lintasan optimasi optimizer, namun tidak dirancang secara spesifik sebagai metode regularisasi utama untuk mengatasi overfitting.

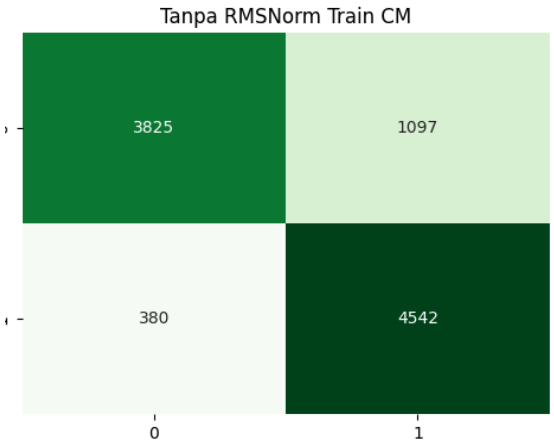


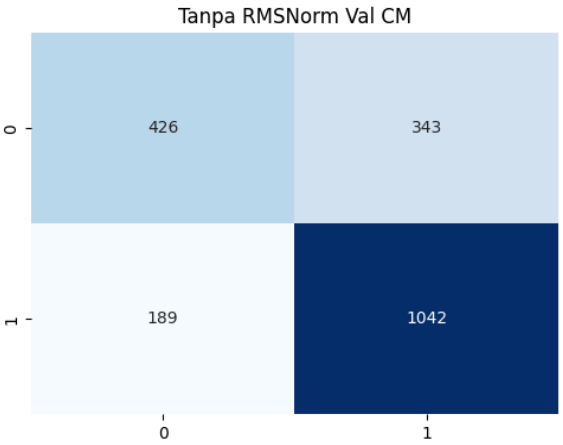


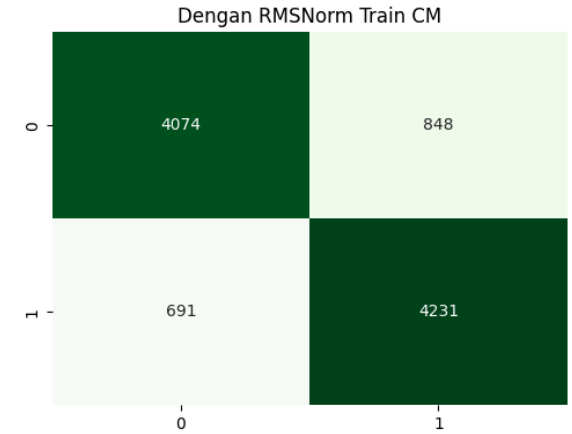


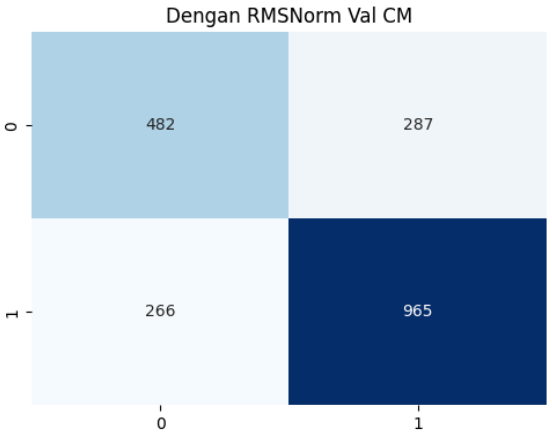


Pada histogram distribusi bobot, kedua kurva tampak mirip dan memiliki pusat di nol, sehingga meskipun RMSNorm memiliki pengaruh ke persebaran bobot dan gradiennya, tapi tidak mengubah magnitude bobot akhir secara drastis. Pada distribusi gradien, gradien pada model dengan RMSNorm lebih terkonsentrasi di sekitar nol dengan memiliki varians gradient yg lebih kecil. Sedangkan, model tanpa RMSNorm memiliki distribusi gradien yang lebih lebar dengan varians yang lebih besar.







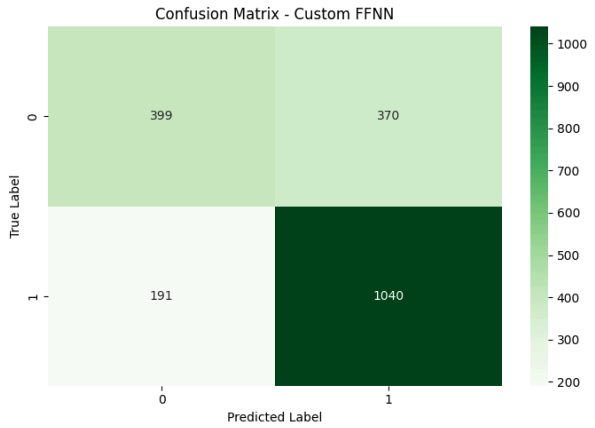


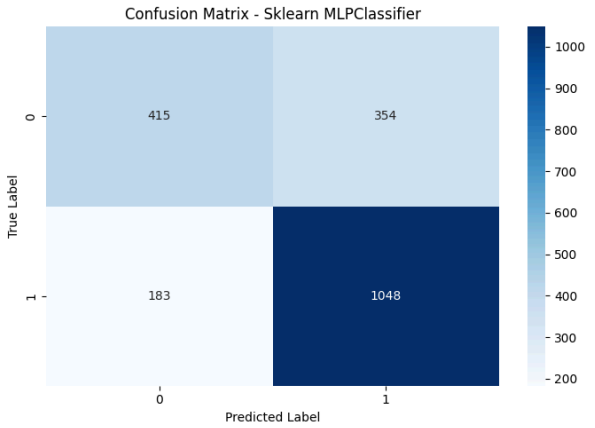
```
EXPERIMENT: Tanpa RMSNorm
ACCURACY | Train: 0.8500 | Validation: 0.7340

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.69      0.55      0.62       769
           1       0.75      0.85      0.80      1231

    accuracy                           0.73      2000
   macro avg       0.72      0.70      0.71      2000
weighted avg       0.73      0.73      0.73      2000
```

```
EXPERIMENT: Dengan RMSNorm
ACCURACY | Train: 0.8437 | Validation: 0.7235

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.64      0.63      0.64       769
           1       0.77      0.78      0.78      1231

    accuracy                           0.72      2000
   macro avg       0.71      0.71      0.71      2000
weighted avg       0.72      0.72      0.72      2000
```

Berdasarkan hasil evaluasi, model yang tidak menggunakan RMSNorm menghasilkan tingkat akurasi validasi yang lebih tinggi (73.40%) dibandingkan model dengan RMSNorm (72.35%), tapi punya bias prediksi yang kuat terhadap kelas mayoritas. Hal ini terlihat pada nilai recall model tanpa normalisasi sangat tinggi pada Kelas 1 (0.85) tetapi gagal mengenali fitur Kelas 0 dengan baik (0.55). Sebaliknya, implementasi RMSNorm pada hidden layer mampu membuat network untuk mempelajari fitur dari kedua kelas secara lebih rata dan bukan hanya menebak kelas mayoritas, yang ditunjukkan oleh peningkatan recall Kelas 0 menjadi 0.63 dan penyesuaian recall Kelas 1 menjadi 0.78, sehingga menghasilkan performa klasifikasi yang jauh lebih seimbang untuk dataset yang memiliki class imbalance.

7. Perbandingan dengan library sklearn.






```
Hasil Prediksi: Custom FFNN
Accuracy: 0.7195
              precision    recall  f1-score   support

           0       0.68      0.52      0.59       769
           1       0.74      0.84      0.79      1231

    accuracy                           0.72      2000
   macro avg       0.71      0.68      0.69      2000
weighted avg       0.71      0.72      0.71      2000
```

```
Hasil Prediksi: Sklearn
Accuracy: 0.7315
              precision    recall  f1-score   support

           0       0.69      0.54      0.61       769
           1       0.75      0.85      0.80      1231

    accuracy                           0.73      2000
   macro avg       0.72      0.70      0.70      2000
weighted avg       0.73      0.73      0.72      2000
```

Pada evaluasi akurasi dan metrik klasifikasi, model Custom FFNN dan Scikit-Learn menghasilkan performa yang mirip, Custom FFNN memiliki akurasi 71.95% dan Sklearn 73.15%. Selisih lainnya seperti precision, recall, dan f1-score juga sangat kecil di range 0.01 sampai 0.02, yang menunjukkan bahwa implementasi pada model custom berjalan dengan baik. Pada bagian confusion matrix, kedua model juga menunjukkan pola bias prediksi yang mirip dalam menangani data imbalanced. Keduanya sama-sama dominan dalam memprediksi kelas mayoritas (Kelas 1) dengan nilai recall 0.84 dan 0.85. Sedangkan, untuk memprediksi kelas minoritas (Kelas 0), kedua model masih kesulitan dengan nilai recall yang lebih rendah

0.52 dan 0.54, sehingga menunjukkan bahwa Custom FFNN punya perilaku pembelajaran yang cukup baik dan sebanding dengan library Scikit-Learn.

Kesimpulan dan Saran

Pada tugas besar ini berhasil mengimplementasikan dan menggunakan modul FFNN yang kustom dan fleksibel. Dari hasil pengujian terlihat bahwa performa modul mendekat penggunaan library *existing* yang tersedia. Pada pengimplementasian modul juga berhasil menerapkan optimasi pembelajaran yang meningkatkan efisiensi dan kualitas model.

Pada pengujian model juga mendemonstrasikan perubahan keluaran berdasarkan perubahan parametrik maupun sub komponen pada model. Mendemonstrasikan pengaruh kompleksitas model terhadap dataset dan faktor penurunan kualitas model seperti *gradient vanishing*. Mendemonstrasikan bagaimana optimasi bobot model dan pengaruhnya pada performa model serta dukungan penghitungan *loss* baik untuk model klasifikasi maupun model regresi

Library ini dapat dikembangkan lebih lanjut untuk meningkatkan efektivitas model dan modularitas library melalui parameter yang lebih flexible khususnya pada format parameter konfigurasi layer. Dapat ditambahkan optimasi pembelajaran agar meningkatkan kualitas keluaran model. Dapat dilakukan eksperimen lebih lanjut dengan library pembanding seperti SKLearn dan Pytorch untuk dapat mengembangkan fitur lebih lanjut.

Pembagian Tugas

Task	NIM
Membuat FFNN	13222061
Data Preprocessing dan Eksperimen	13222033
Laporan	13222033,13222061

Bonus Yang Dikerjakan

- (5% total bonus) Implementasikan minimal 2 fungsi aktivasi lain yang sering digunakan.
- (5% total bonus) Implementasikan 2 metode inisialisasi bobot berikut:
    - ○ Xavier initialization.
    - ○ He initialization
- (10% total bonus) Implementasikan metode normalisasi RMSNorm, kemudian lakukan analisis untuk beberapa hal berikut:
    - Lakukan eksperimen masing-masing 1 kali untuk model tanpa normalisasi dan dengan normalisasi.
    - Bandingkan hasil akhir prediksinya.
    - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
    - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
- (40% total bonus) Implementasikan optimizer Adam (Adaptive Moment Estimation) sebagai alternatif dari Gradient Descent standar dan lakukan analisis perbandingan kecepatan konvergensi antara Gradient Descent standar dengan Adam pada dataset yang diberikan.

Referensi

[1] Tugas Besar 1 IF3270 Pembelajaran Mesin Feedforward Neural Network, Tim Asisten IF3270 2025/2026

[2] Slide Kuliah IF3270

[3] https://www.geeksforgeeks.org/machine-learning/leaky-relu-activation-function-in-deep-learning/

[4] https://www.geeksforgeeks.org/deep-learning/elu-activation-function-in-neural-network/

[5] Root Mean Square Layer Normalization, Biao Zhang, Rico Sennrich (https://openreview.net/pdf?id=SygkZ3MTJE)

[6] ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION (https://arxiv.org/pdf/1412.6980)

[7] https://www.geeksforgeeks.org/deep-learning/vanishing-and-exploding-gradients-problems-in-deep-learning/

[8] https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/

[9] https://www.geeksforgeeks.org/machine-learning/impact-of-learning-rate-on-a-model/

[10] https://www.geeksforgeeks.org/machine-learning/regularization-in-machine-learning/

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : William Gerald Briandolo / 132 22 061
Fakultas/Sekolah : STEI
Kode Mata Kuliah : IF 3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 1 – FFNN

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/~~Tidak~~*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Jakarta, 18 Maret 2026

William Gerald Briandolo
13222061

Created by Notein

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : Anas Fathurrahman
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Rekayasa
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Laporan Tugas Besar 1 IF3270 Pembelajaran Mesin Semester II Tahun Akademik 2025/2026 Implementasi Algoritma Pembelajaran Mesin

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/Tidak*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/Tidak*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/Tidak*	Diskusi mendalam membahas kemungkinan teoritis.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/Tidak*	
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18 Maret 2026

Anas Fathurrahman, 13222033